

Aerospace Predictive Maintenance

Version 2 — Production-Oriented Architecture

This document describes **src_v2/**, a production-oriented evolution of the original hackathon MVP. It covers every new file and function, the bigger training dataset, the from-scratch Isolation Forest and Random Forest implementations and why they were chosen, the train/serve architecture, model registry, benchmark comparison against v1, and remaining limitations.

This is a new, separate document. The original `code_documentation.pdf` and `README.md` continue to describe the original `src/` MVP exactly as before and were not modified.

Pipeline: `train.py` (offline) → simulate larger fleet → engineer rolling-window features → train Isolation Forest + Random Forest per component → tune alert thresholds on a validation split → benchmark on a held-out test split → save artifacts + model card. `predict_service.py` (serving) → loads saved artifacts → scores new telemetry → builds dashboard → sends work orders.

What Changed: Production-Readiness Improvements

Aspect	v1	v2
Fleet size	8 aircraft, 32 parts, ~3,000 rows	40 aircraft, 160 parts, ~16,300 rows (5x)
Sensor channels	3 (vibration, temperature, pressure)	5 (adds oil debris, RPM deviation)
Features fed to models	Raw instantaneous sensor readings only	31 features: raw + rolling mean/std/min/max/slope per sensor
Anomaly detection algorithm	Per-unit z-score threshold (statistics)	Isolation Forest per component (ML ensemble, from scratch)
RUL estimation algorithm	Linear regression per component	Random Forest Regressor per component (from scratch)
Data split	Train / test, by unit	Train / validation / test, by unit (validation tunes alert thresholds)
Model lifecycle	Retrained inside main.py every run	train.py (offline, occasional) vs. predict_service.py (serving, frequent) - separated
Model persistence	None - nothing saved to disk	artifacts/*.pkl (pickled models) + model_card.json (version, hyperparameters, metrics, timestamp)
Alert thresholds	Hand-picked constant (Z_SCORE_LIMIT=4.0)	Learned from validation-split score quantiles (target contamination rate)
Automated tests	None	tests/test_models.py - 7 unittest cases covering both models + feature engineering
MRO integration robustness	Fire-and-forget mock send	Idempotency key per work order + simulated retry/backoff
Configuration	Constants scattered across files	Centralized config.py

File-by-File Documentation

config.py

Central configuration module: fleet size, sensor list, feature-engineering window, Isolation Forest and Random Forest hyperparameters, train/val/test split fractions, dashboard thresholds, and file paths. No functions - just named constants, so retraining with different settings never requires touching model code.

data_simulator_v2.py

Generates the larger, 5-sensor synthetic fleet and the explicit train/val/test unit split.

Function	Inputs	Output	What it does
<code>make_one_unit_data(rng, aircraft_id, component, unit_id)</code>	<code>rng</code> : numpy Generator <code>aircraft_id</code> : component, <code>unit_id</code> : str	DataFrame: one row per simulated cycle, 5 sensor columns + RUL, <code>true_anomaly</code> , <code>life_length</code>	Same wear-curve simulation approach as v1, extended to 5 sensor channels (adds <code>oil_debris</code> , <code>rpm_deviation</code> , both rising with wear).
<code>simulate_fleet(seed=42)</code>	<code>seed</code> : int	DataFrame: all units stacked (<code>N_AIRCRAFT</code> × 4 components = 160 units)	Builds the full 40-aircraft fleet.
<code>split_units(fleet_df, val_fraction, test_fraction, seed)</code>	<code>fleet_df</code> : DataFrame <code>val_fraction</code> , <code>test_fraction</code> : float <code>seed</code> : int	DataFrame: <code>unit_id</code> -> split ('train'/'val'/'test')	Production-style 3-way split by <code>unit_id</code> : train fits the models, validation tunes the anomaly alert thresholds, test gives a final untouched accuracy check.

features.py

Converts raw sensor readings into the 31-column feature set used by BOTH models: the raw instantaneous reading (keeps single-cycle spikes visible) plus rolling mean/std/min/max/slope over a trailing 5-cycle window (captures the slower accelerating-wear trend).

Function	Inputs	Output	What it does
<code>_slope(values)</code>	<code>values</code> : 1D numpy array (a rolling window of one sensor)	float: least-squares slope of the window	Fits a straight line through the trailing window and returns its slope - a rising slope flags developing wear even before the absolute value looks unusual.
<code>add_rolling_features(df, window=5)</code>	<code>df</code> : DataFrame sorted by <code>unit_id</code> , cycle window: int	same DataFrame + 5 new columns per sensor (mean/std/min/max/slope)	Computes trailing rolling statistics per <code>unit_id</code> per sensor.
<code>feature_column_names()</code>	None	list of 31 column names, in a fixed order	The single source of truth for which columns feed the models - both <code>train.py</code> and <code>predict_service.py</code> import this so training and serving can never drift apart.

models/isolation_forest.py

From-scratch implementation of Isolation Forest (Liu, Ting & Zhou, 2008), the same algorithm behind scikit-learn's IsolationForest. See the 'Why These Algorithms' section for why it was reimplemented rather than imported.

Function	Inputs	Output	What it does
<code>_average_path_length(n)</code>	n: int, subsample size	float: expected BST search path length	Normalization constant $c(n)$ from the original paper, used to turn a raw path length into a 0-1 anomaly score.
<code>_IsolationTree.fit(X, depth, rng)</code>	X: 2D numpy array depth: int rng: numpy Generator	None (builds the tree in place)	Recursively splits on a random feature at a random threshold until max depth or an unsplittable node is reached.
<code>_IsolationTree.path_length(x)</code>	x: 1D numpy array, one row	float: path length (+ correction) for that point	Walks a single point down the tree and returns how many splits it took to isolate it.
<code>IsolationForest.fit(X)</code>	X: 2D numpy array, shape (n_samples, n_features)	self (fitted forest)	Builds <code>n_trees</code> isolation trees, each on an independent random subsample (without replacement) of size <code>subsample_size</code> .
<code>IsolationForest.score(X)</code>	X: 2D numpy array	1D numpy array: anomaly score per row (near 1.0 = anomalous)	Averages each point's path length across all trees and converts it to the standard $2^{(-avg_path/c(\psi))}$ anomaly score.

models/random_forest.py

From-scratch implementation of a Random Forest Regressor - bootstrap-aggregated (bagged) decision trees with feature bagging and histogram-based (quantile) split finding for speed, the same algorithm family behind scikit-learn's RandomForestRegressor.

Function	Inputs	Output	What it does
<code>_DecisionTreeRegressor.fit(X, y, depth, rng)</code>	X: 2D numpy array y: 1D numpy array (targets) depth: int rng: numpy Generator	None (builds the tree in place)	At each node, samples a random subset of features, tries quantile-based candidate thresholds for each, and picks the (feature, threshold) that most reduces squared error; recurses until max depth / min-samples limits are hit.
<code>_DecisionTreeRegressor.predict(X)</code>	X: 2D numpy array	1D numpy array: predicted value per row	Walks each row down the tree to a leaf and returns that leaf's mean training value.
<code>RandomForestRegressor.fit(X, y)</code>	X: 2D numpy array y: 1D numpy array	self (fitted forest)	Builds <code>n_trees</code> decision trees, each on an independent bootstrap sample (with replacement) of the training data - the 'bagging' that makes a random forest more stable than one tree.
<code>RandomForestRegressor.predict(X)</code>	X: 2D numpy array	1D numpy array: predicted RUL per row	Averages every tree's prediction.
<code>RandomForestRegressor.feature_importances_(n_features)</code>	n_features: int	1D numpy array: normalized importance per feature index	Counts how often each feature was used as a split point across all trees, as a simple, standard importance proxy.

anomaly_detection_v2.py

Function	Inputs	Output	What it does
<code>train_anomaly_models(featured_train_df, engineered_train_df)</code>	featured_train_df: engineered features, TRAIN split only	dict {component: fitted IsolationForest}	Trains one Isolation Forest per component type on the 31-column feature set.
<code>compute_thresholds(models, featured_val_df, contamination=0.05)</code>	models: dict from train_anomaly_models featured_val_df: VALIDATION split contamination: float	dict {component: threshold score}	Scores the validation split and picks each component's alert threshold as the score quantile matching the target contamination rate - a data-driven replacement for a hand-picked constant.
<code>score_anomalies(models, thresholds, featured_df)</code>	models, thresholds: from the two functions above featured_df: any engineered-feature DataFrame	same DataFrame + anomaly_score, anomaly_threshold, is_anomaly columns	Scores every row with its component's model and flags it if the score exceeds that component's tuned threshold.

rul_model_v2.py

Function	Inputs	Output	What it does
<code>train_rul_models(featured_train_df)</code>	featured_train_df: TRAIN split	dict {component: fitted RandomForestRegressor}	Trains one Random Forest per component type to predict RUL from the 31-column feature set.
<code>predict_rul(models, df)</code>	models: dict from train_rul_models df: engineered-feature DataFrame	1D numpy array: predicted RUL per row (clipped at 0)	Routes each row to the right component's model.
<code>evaluate(y_true, y_pred)</code>	y_true, y_pred: numpy arrays	dict {'MAE', 'RMSE'}	Standard regression accuracy metrics.
<code>evaluate_by_component(df_with_predictions)</code>	df_with_predictions: DataFrame with RUL + predicted_RUL columns	dict {component: {'MAE','RMSE','n_rows'}}	Breaks accuracy down per component type.

evaluation_v2.py

Same maintenance-relevant benchmark metrics as v1's evaluation.py (detection lead time, two-sided false alarm rate, per-component RUL accuracy), recomputed for the v2 models and dataset. See the v1 documentation for the detailed reasoning behind each metric's design; the formulas are unchanged.

Function	Inputs	Output	What it does
<code>evaluate_anomaly_detection(scored_df)</code>	scored_df: fleet data with is_anomaly, true_anomaly, RUL, life_length columns	dict: precision, recall, false_alarm_rate, early_life_false_alarm_rate, mean/median detection lead time, units flagged	Same logic as v1: two false-alarm-rate variants (whole-life vs. early-life-only) plus lead time measured against each part's true uncapped remaining life.
<code>evaluate_rul_by_component(df_with_predictions)</code>	df_with_predictions: DataFrame with component, RUL, predicted_RUL	dict {component: {'MAE','RMSE','n_rows'}}	Per-component RUL accuracy breakdown.

Function	Inputs	Output	What it does
<code>evaluate_rul_overall(df_with_predictions)</code>	<code>df_with_predictions:</code> DataFrame with RUL, <code>predicted_RUL</code>	dict { <code>'MAE'</code> , <code>'RMSE'</code> , <code>'n_rows'</code> }	Pooled RUL accuracy across all components.

model_registry.py

A minimal model registry: saves/loads the trained model objects and maintains a JSON 'model card' recording what was trained, when, with which hyperparameters, and how well it scored - the metadata a real MLOps setup would track to answer 'which model is currently deployed, and can we trust it?'.

Function	Inputs	Output	What it does
<code>save_models(models_by_component, name, artifact_dir, hyperparameters, metrics, extra)</code>	<code>models_by_component</code> : dict {component: model} <code>name</code> : str ('iforest' or 'rf') <code>artifact_dir</code> : str <code>hyperparameters</code> , <code>metrics</code> , <code>extra</code> : dict	None (writes .pkl files + updates <code>model_card.json</code>)	Pickles each component's model to its own file and records a model-card entry (timestamp, hyperparameters, metrics, any extra data such as tuned thresholds).
<code>load_models(name, components, artifact_dir)</code>	<code>name</code> : str <code>components</code> : list of str <code>artifact_dir</code> : str	dict {component: unpickled model}	Loads back exactly what <code>save_models</code> wrote - used by <code>predict_service.py</code> so serving never has to retrain.
<code>load_model_card(artifact_dir)</code>	<code>artifact_dir</code> : str	dict (empty if no card exists yet)	Reads the JSON model card, e.g. to display 'model last trained at ...' or recover saved thresholds.

dashboard_v2.py

Same fleet dashboard concept as v1, with a performance panel labeled for the v2 models and a finer anomaly-score display (3 decimal places, since Isolation Forest scores are a 0-1 probability-like value rather than a z-score).

Function	Inputs	Output	What it does
<code>classify_health(row)</code>	<code>row</code> : Series with <code>predicted_RUL</code> , <code>is_anomaly</code>	str: 'Critical' / 'Warning' / 'Healthy'	Same threshold logic as v1 (CRITICAL_RUL=25, WARNING_RUL=60, now read from <code>config.py</code>).
<code>build_dashboard(scored_df)</code>	<code>scored_df</code> : fleet data with <code>predicted_RUL</code> , <code>anomaly_score</code> , <code>is_anomaly</code>	DataFrame: one row per unit (latest cycle), sorted most-urgent-first	Same logic as v1's <code>build_dashboard</code> , simplified signature since v2 already has <code>predicted_RUL</code> attached before this is called.
<code>save_dashboard_html(dashboard_df, path, metrics_summary=None)</code>	<code>dashboard_df</code> : DataFrame <code>path</code> : str <code>metrics_summary</code> : optional dict	None (writes .html)	Renders the color-coded fleet table plus a 'Model Performance Benchmarks (v2)' panel sourced from <code>model_metrics_v2.json</code> .

mro_integration_v2.py

Function	Inputs	Output	What it does
<code>build_work_order(row)</code>	<code>row</code> : one dashboard row	dict: adds <code>anomaly_score</code> and uses UTC timestamps vs. v1	Same fields as v1 plus the numeric anomaly score, useful for a human reviewing why a work order was raised.
<code>create_work_orders(dashboard_df)</code>	<code>dashboard_df</code> : fleet dashboard table	list of dicts (Critical/Warning rows only)	Unchanged from v1's logic.

Function	Inputs	Output	What it does
<code>send_to_mro_system(work_orders, path, max_retries, failure_rate, seed)</code>	<code>work_orders</code> : list of dicts <code>path</code> : str <code>max_retries</code> : int <code>failure_rate</code> : float (0 by default) <code>seed</code> : int	tuple (sent, failed) lists; writes JSON file	Wraps each send in a retry loop (simulating a flaky network call to a real MRO API); work orders that exhaust all retries are returned separately (would route to a dead-letter queue in production) instead of silently vanishing.

train.py (offline training entry point)

Run occasionally, whenever there is new historical data to retrain on. Generates the dataset, engineers features, trains both models per component, tunes anomaly thresholds on the validation split, benchmarks everything on the untouched test split, and saves models + model card + a benchmark report. Typical runtime in this sandbox: ~35-40 seconds.

Function	Inputs	Output	What it does
<code>main()</code>	None	None. Side effects: writes <code>data_v2/fleet_telemetry_v2.csv</code> , <code>artifacts/*.pkl</code> , <code>artifacts/model_card.json</code> , <code>outputs_v2/model_metrics_v2.json</code>	Orchestrates the full offline training pipeline end-to-end, printing progress at each stage.

predict_service.py (production serving entry point)

Run frequently (e.g. on a schedule, or whenever new telemetry lands). Loads the artifacts train.py already produced - no retraining - scores a fresh telemetry batch, builds the dashboard, and sends work orders. Typical runtime in this sandbox: ~13 seconds, roughly 3x faster than train.py, which is the point of separating the two.

Function	Inputs	Output	What it does
<code>load_new_telemetry_batch(seed=99)</code>	<code>seed: int</code>	DataFrame: a fresh simulated fleet snapshot	Stands in for 'new telemetry arriving after deployment' - a different random seed than train.py used, so the model is genuinely scoring data it has not seen.
<code>main()</code>	None	None. Side effects: writes <code>outputs_v2/dashboard_v2.html</code> , <code>fleet_status_v2.csv</code> , <code>work_orders_v2.json</code>	Loads saved models -> fetches new telemetry -> engineers features -> scores anomalies + predicts RUL -> builds dashboard -> creates and sends work orders.

tests/test_models.py

Seven unittest-based sanity tests (no pytest dependency required) verifying: Isolation Forest scores injected outliers higher than inliers and stays within its expected 0-1 range; Random Forest fits a known linear relationship within a reasonable error tolerance and correctly ranks an informative feature above an irrelevant one; and the rolling-slope feature has the correct sign on rising vs. flat series. All 7 pass (`python -m unittest tests.test_models -v`).

Why Isolation Forest and Random Forest (Production-Grade Algorithms)

A note on implementation: this sandbox has no outbound internet access, so scikit-learn could not be installed via pip (verified: pip install fails with a 403 from the network proxy). Both algorithms below were therefore reimplemented from scratch in plain numpy, matching the published algorithms and scikit-learn's own semantics. The code is structured so that swapping in sklearn.ensemble.IsolationForest / RandomForestRegressor later, in an environment with normal internet access, is a drop-in replacement behind the same fit/predict/score calls used throughout this project.

Isolation Forest for anomaly detection

- **Purpose-built for anomaly detection, not adapted from something else:** unlike z-score (v1), which only reasons about one dimension at a time, Isolation Forest reasons over the full multi-sensor feature vector at once, catching anomalies that only show up as an unusual COMBINATION of readings rather than any single sensor being extreme.
- **No distributional assumptions:** z-score implicitly assumes roughly normal, stable variance data; Isolation Forest makes no such assumption, which matters once 31 engineered features with very different distributions are all being scored together.
- **Industry-standard, well understood:** Isolation Forest is one of the most widely deployed unsupervised anomaly detectors in production systems (fraud detection, infrastructure monitoring, and industrial IoT), so choosing it - even reimplemented - keeps the design decision itself defensible and familiar to anyone auditing the system.
- **Threshold tuned on data, not guessed:** pairing it with a validation-split-derived contamination threshold (compute_thresholds) turns 'how suspicious is suspicious enough' into a data-driven decision instead of a hand-picked constant.
- **Same upgrade path as before:** the class exposes fit(X) / score(X), identical to scikit-learn's IsolationForest.fit / .score_samples contract, so this can be swapped for the library version with no changes to anomaly_detection_v2.py.

Random Forest for Remaining Useful Life

- **Captures non-linear, accelerating wear:** v1's linear regression assumes a straight-line relationship between sensor readings and RUL; real wear accelerates near end-of-life. A tree ensemble naturally partitions the feature space into regions with different local behavior, fitting this curve far better - reflected in the roughly 3x lower MAE reported below.
- **Scale-invariant by construction:** tree splits only ask 'is this feature above or below a threshold', so mixing engine temperature (~550) with landing gear pressure (~3000) in the same feature vector causes no distortion the way it would for linear regression - useful now that all 31 engineered features are fed to one model per component.
- **Bagging reduces overfitting risk:** each of the 25 trees per component trains on an independent bootstrap resample with a random feature subset per split, and predictions are averaged - a single decision tree would badly overfit a dataset this size, but the ensemble is far more stable.
- **Feature importances come for free:** feature_importances() gives a ranked view of which sensors/statistics actually drive each component's RUL prediction, letting engineers sanity-check the model in a way a linear model's raw coefficients can approximate but a deep network cannot.
- **Histogram-based splits keep training fast without sklearn:** testing every possible split threshold would be far too slow in pure Python; using quantile-based candidate thresholds (the same 'binning' trick production gradient-boosting libraries like LightGBM use) keeps training all 4 components under 40 seconds even in this sandbox.

Benchmark Comparison: v1 vs v2

Figures are from representative runs of each version's own benchmark scripts (v1: main.py; v2: train.py), evaluated on each version's own held-out test data. Exact numbers vary run to run since the fleets are randomly generated.

Aspect	v1	v2
Fleet / rows used for benchmarking	32 parts, ~3,000 rows (whole fleet scored)	160 parts, 16,288 rows total; test split = 24 units / ~2,700 rows
RUL MAE - overall	~13.5 cycles	~4.1 cycles
RUL MAE - engine	~4.1 cycles	~1.4 cycles
RUL MAE - brakes	~4.2 cycles	~2.8 cycles
RUL MAE - bogie	~5.6 cycles	~6.7 cycles (comparable)
RUL MAE - landing gear (v1's weak point)	~20.8 cycles	~5.0 cycles
Anomaly recall	100% (6/6 injected spikes, whole fleet)	67% (2/3 injected spikes, small test split - see caveat below)
Early-life false alarm rate	0.0%	4.6%
Mean detection lead time	~150 cycles	~196 cycles

Caveat on anomaly metrics: v1's recall/false-alarm figures were computed by scoring the entire simulated fleet (no held-out split existed for the detector). v2's figures are computed correctly on a genuinely held-out TEST split, which is the right way to measure it - but that split only contains 24 of the 160 units, and only 3 of those happened to have an injected anomaly. A single miss therefore swings recall by 33 percentage points. The RUL and lead-time improvements are measured on much larger samples (hundreds to over a thousand rows per component) and are far more statistically reliable than the anomaly precision/recall figures.

Limitations and Next Steps

Both models are still trained entirely on synthetic data. The Isolation Forest / Random Forest implementations, while algorithmically faithful, are not vectorized as heavily as scikit-learn's Cython/C implementations, so hyperparameters (25 trees, depth 6, 8 histogram bins) were tuned down from typical defaults to keep training under a minute in this sandbox - a real deployment with proper infrastructure (or working internet access to install scikit-learn directly) could use larger ensembles for further accuracy gains. The 3-unit anomaly test sample noted above should be grown (larger fleet, or a dedicated larger anomaly-only evaluation set) before quoting precision/recall with confidence. Before production use: retrain and validate against real sensor and maintenance-outcome data, and connect mro_integration_v2.py to a real system per the roadmap already documented in code_documentation.pdf (still applicable unchanged to v2).